Security & Maintenance Report

Orator - Neural Speech Analysis (Speaker Hub)

Next.js 14 (App Router) + Supabase + OpenAI. This report documents a full security review of the website and the Supabase backend, every change applied in this pass, verification results, and the remaining recommendations the team should action.

Date	2026-06-24
Repository	mickhatzikostas220-design/speech-analyzer2
Branch	claude/compassionate-brahmagupta-i55gpm
Supabase ref	tmjyujpprveeqhorjcgt
Scope	Site (API routes + libs) and Supabase (RLS, advisors, auth config)

At a glance

- 9 code fixes applied across 11 files; build + typecheck green.
- 2 Critical, 2 High, and 5 lower-severity issues found and fixed.
- Supabase: RLS enabled on all 24 tables (verified). 1 auth-config warning.
- Dependencies: transitive vulns auto-patched; 1 Next.js bump recommended.

1. Executive summary

A focused security audit of the Orator codebase and its Supabase backend surfaced two Critical and two High severity issues, plus several lower-severity hardening gaps. All of them have been fixed in this pass. The most important findings were a forgeable admin-approval token (a hardcoded fallback signing secret) and a cross-tenant file-read path in the editor, where client-controlled storage paths were signed by the service-role client without an ownership check.

On the backend, Supabase is in good shape: Row Level Security is enabled on all 24 public tables, OAuth and API-key secrets are encrypted at rest with AES-256-GCM, and no secret is ever selected into a browser response. The single outstanding Supabase item is an auth configuration warning (leaked-password protection is disabled), which is a dashboard toggle.

Every change was verified with a clean TypeScript typecheck and a successful production build. The work was developed on the feature branch above and is ready for review.

Severity tally

Severity	Found	Status
Critical	2	Fixed
High	2	Fixed
Low / Hardening	5	Fixed
Supabase advisor	1	Documented (dashboard toggle)
Dependency (npm)	13	Transitive auto-fixed; 1 major bump advised

2. Changes applied

Each entry lists the severity, the file(s) touched, the problem, and the fix.

CRITICAL

Forgeable admin approve/deny token

lib/adminToken.ts (used by app/api/admin/action/route.ts)

Problem

The HMAC signing secret fell back to a hardcoded literal ('fallback-secret-change-me') when ADMIN_ACTION_SECRET was unset. The /api/admin/action route authenticates SOLELY on this token (no session check), so with the env var missing an attacker could forge a valid 'approve' token for any pending access request and self-grant an invite - bypassing the entire invite-only allowlist.

Fix

Removed the fallback: secret() now throws if ADMIN_ACTION_SECRET is unset or shorter than 16 chars (fail closed). Signature comparison switched to crypto.timingSafeEqual. Documented ADMIN_ACTION_SECRET as required in .env.local.example.

CRITICAL

Cross-tenant file read via client-controlled clip paths

app/api/editor/timeline/[id]/route.ts, app/api/editor/script/[id]/route.ts

Problem

On GET, these routes minted signed Storage URLs for clip paths taken from project JSON that the owner can freely PATCH. A user could insert another user's storage path (e.g. victim-uuid/editor/.../original.mp4) into their own project and receive a working 1-hour signed URL via the RLS-bypassing service-role client - reading any file in the 'speeches' bucket. The script DELETE path had the mirror problem (deleting another user's files).

Fix

Added an ownership guard: only paths beginning with `${user.id}/` are ever signed (GET) or removed (DELETE). All storage uploads already use that prefix, so legitimate paths are unaffected.

HIGH

SSRF in brand extraction (user-supplied website URL)

lib/brand/extract.ts (+ new lib/net/safeFetch.ts)

Problem

Brand auto-extraction does a server-side fetch of a user-entered URL with only a scheme/TLD check and redirect:'follow'. An authenticated user could target http://169.254.169.254/ (cloud metadata), localhost, or an external host that 302-redirects there, and have the server make the request from inside the trust boundary - with extracted text reflected back in the response.

Fix

Introduced a shared SSRF-hardened fetch (lib/net/safeFetch.ts) that allows only http/https on ports 80/443, resolves the hostname and rejects private/loopback/link-local/ULA/CGNAT/multicast addresses, and follows redirects manually - re-validating every hop. Wired it into brand extraction; SsrfError surfaces a clean message.

HIGH

SSRF in calendar (ICS) connect

lib/gigs/ics.ts

Problem

Connecting a calendar fetches the user-supplied iCal/webcal URL server-side with no private-range or redirect protection - the same SSRF class as brand extraction, usable for blind internal host/port probing.

Fix

Routed the ICS fetch through the same safeFetch guard. The function already returns [] on failure, so a blocked URL fails closed and silently.

LOW

Unauthenticated OpenAI endpoint (cost/abuse)

app/api/compare-report/route.ts

Problem

The compare-report route spent GPT-4o tokens with no authentication (middleware does not cover /api). Anyone could call it repeatedly and run up the OpenAI bill.

Fix

Added a supabase.auth.getUser() gate (401 when signed out) plus basic input validation, and made the OpenAI client lazy so a missing key returns 500 instead of throwing at module load (which also unblocked the build).

LOW

Non-constant-time CRON_SECRET comparison

app/api/clipflow/jobs/run/route.ts

Problem

The cron bearer token was compared with '===', a theoretical timing oracle.

Fix

Replaced with a length-checked crypto.timingSafeEqual comparison.

LOW

Command-injection hardening at the yt-dlp boundary

lib/clipflow/clipper.ts

Problem

The YouTube id is interpolated into a shell command run via exec. It is normally regex-parsed or API-sourced, but there was no allowlist at the exec site itself.

Fix

Validate youtubeId against ^[A-Za-z0-9_-]{6,20}$ before building the command; reject otherwise (defense in depth).

LOW

Defense-in-depth user scoping on Gmail token lookup

lib/agent/google.ts, lib/agent/tools/gmail.ts, lib/agent/tools/registry.ts

Problem

getValidAccessToken fetched an agent_connection by id only, using the service-role (RLS-bypassing) client. Not currently exploitable (the id always comes from a user-scoped list), but a latent IDOR if a future caller passes a request-supplied id.

Fix

Threaded userId through and added .eq('user_id', userId) to both the select and the token-refresh update.

MAINT

Dependency vulnerabilities (npm audit)

package-lock.json

Problem

npm audit reported 14 vulnerabilities (9 high), including a high-severity Next.js request-deserialization DoS and several transitive issues (ws, form-data, minimatch, postcss, uuid, glob).

Fix

Ran the non-breaking `npm audit fix`, which patched the transitive items via the lockfile (no package.json/runtime change). The remaining items require Next.js 16 (a major upgrade) and are left as a tracked recommendation rather than an autonomous breaking change.

3. Supabase backend review

Project ref tmjyujpprveeqhorjcgt was reviewed live via the Supabase MCP tooling: table inventory, RLS status, and the security/performance advisors.

Row Level Security

RLS is ENABLED on all 24 public tables. Verified table list:

profiles, analyses, feedback_points, engagement_timeline, access_requests, editor_projects, script_projects, timeline_projects, agent_settings, agent_api_keys, agent_connections, agent_conversations, agent_messages, agent_actions, gigs, bookings, clipflow_projects, clipflow_clips, clipflow_connections, clipflow_posts, clipflow_jobs, clipflow_uploadpost_keys, aeo_settings, aeo_tips.

API routes that use the service-role client (which bypasses RLS) were checked individually; they consistently scope queries by user_id, and the two places that did not (editor signed-URL minting) are fixed in this pass.

Security advisor

WARN

Leaked password protection disabled

Supabase Auth can reject passwords known to be compromised (HaveIBeenPwned). It is currently off. This is an Authentication settings toggle, not a code change.

Remediation: Dashboard -> Authentication -> Policies -> enable 'Leaked password protection'. https://supabase.com/docs/guides/auth/password-security

Performance advisor

11 INFO-level 'unused index' notices (indexes that have not been used yet). These are expected on a young/low-traffic database and are not a problem; revisit once real usage accrues before dropping any. No action taken.

Secrets at rest

- Agent BYOK API keys + OAuth tokens: AES-256-GCM (lib/crypto.ts).
- ClipFlow platform OAuth tokens: AES-256-GCM (lib/clipflow/crypto.ts).
- Verified: no encrypted_* column is ever selected into a client response.

4. Verification

TypeScript

```
$ npx tsc --noEmit
-> exit 0 (no errors)
```

Production build

```
$ npm run build
-> Compiled successfully; types valid; 41 pages generated.
```

Note: the build requires NEXT_PUBLIC_SUPABASE_URL / ANON_KEY and OPENAI_API_KEY to be present (as they are on Vercel). With those set the build completes cleanly end to end; this was confirmed in the sandbox with placeholder values.

Scope of testing

Changes were validated by static typecheck and a full build. No runtime end-to-end test was run against live Supabase/OpenAI from the sandbox. The fixes are localized and behavior-preserving for legitimate requests; a smoke test of admin approval, brand extraction, calendar connect, and the editor after deploy is recommended.

5. Remaining recommendations

Not done in this pass - they need product decisions, env/dashboard access, or a breaking upgrade.

1. Set ADMIN_ACTION_SECRET in every environment

Now REQUIRED (the insecure fallback was removed). Generate with `openssl rand -hex 32` and add it to Vercel + local env, or the admin email approve/deny links will stop working.

2. Enable leaked-password protection in Supabase

One dashboard toggle (see section 3).

3. Plan a Next.js upgrade

A high-severity Next.js DoS advisory is only resolved by moving off 14.2.x (audit fix proposes 16.x, a major bump). Schedule and test this deliberately rather than auto-applying.

4. Add rate limiting to API routes

Already a known limitation in the README. The transcription, AI-copy, compare-report, and brand/ICS-fetch routes are the priority targets (cost + SSRF surface). Upstash Ratelimit or similar.

5. Initiate OAuth connects via POST, not GET

The Gmail and ClipFlow 'connect' flows start on a GET link, allowing forced-connect CSRF. Impact is low (victim connects their own account), but a POST + CSRF token closes it.

6. Verify project ownership on editor upload routes

The signed-upload routes scope writes to the caller's own `${user.id}/` prefix (so no cross-tenant write), but do not confirm the project id belongs to the caller. Add .eq('user_id', user.id) for tidiness/quota.

Files changed in this pass: .env.local.example, app/api/clipflow/jobs/run/route.ts, app/api/compare-report/route.ts, app/api/editor/script/[id]/route.ts, app/api/editor/timeline/[id]/route.ts, lib/adminToken.ts, lib/agent/google.ts, lib/agent/tools/gmail.ts, lib/agent/tools/registry.ts, lib/brand/extract.ts, lib/clipflow/clipper.ts, lib/gigs/ics.ts, lib/net/safeFetch.ts (new), package-lock.json.